Shopizer E-Commerce Platform

Part 6. Static Code Analysis

Course: Software Testing
Student: Yijun Sun, Yu Chien Chiu
Date: March 9, 2026

1. What Are Static Analyzers?

Static analysis tools examine source code without executing it, detecting potential bugs, security vulnerabilities, and code quality issues at development or CI/CD time. Unlike dynamic testing, static analyzers work on the codebase itself — inspecting control flow, data flow, type usage, and coding patterns — making them fast and deterministic.

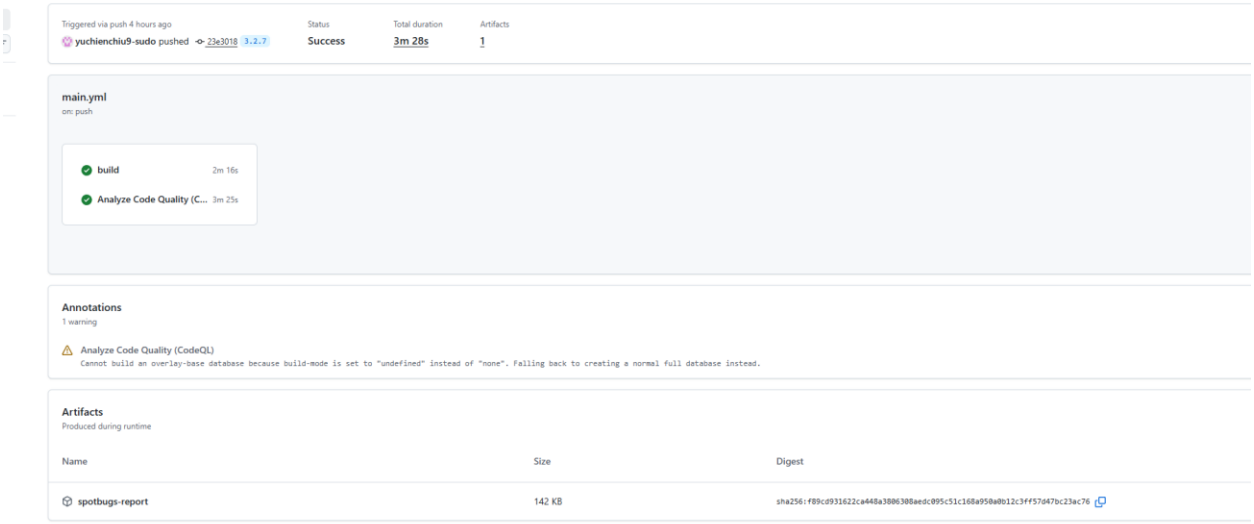
The main goals and uses of static analyzers are:

- Early bug detection: Catch defects such as null pointer dereferences, type mismatches, and resource leaks before the code is ever run.
- Security hardening: Identify common vulnerability patterns (OWASP Top 10, CWE) like path traversal, injection, and broken cryptography.
- Code quality enforcement: Flag bad practices such as dead code, ignored return values, or non-portable encoding assumptions.
- CI/CD integration: Run automatically on every pull request or commit so developers get instant feedback without manual code review.
- Compliance: Help teams meet security standards (e.g., OWASP, CERT, CWE) by systematically checking for known weakness patterns.

2. Tools Used in This Project

Two static analyzers were integrated into the GitHub Actions CI/CD pipeline for the Shopizer project:

Tool	Type	Integration	Findings
GitHub CodeQL	Semantic / data-flow analysis	GitHub Actions + Security tab	45 findings
SpotBugs 4.7.3.6	Bytecode / pattern matching	Maven plugin + artifact upload	160 findings (sm-core)



3. SpotBugs: Findings Overview

SpotBugs analyzed the sm-core module (346 classes, 13,098 lines) and found 160 bugs: 24 Priority 1 (high) and 136 Priority 2 (medium).

Category	Bug Count	Severity	Key Pattern
CORRECTNESS	~20	Priority 1	Logic errors — equals() type mismatch, ignored BigDecimal results
I18N	~8	Priority 1	Default platform encoding in crypto and I/O
MALICIOUS_CODE	~40	Priority 2	Mutable object exposure via getters/setters
BAD_PRACTICE	~15	Priority 2	Non-serializable fields, ignored File.delete()
PERFORMANCE	~18	Priority 2	keySet() iterator, String concat in loop
STYLE / DODGY	~59	Priority 2	Dead stores, vacuous instanceof, static write from instance

3.1 Detailed SpotBugs Finding: RV_RETURN_VALUE_IGNORED (BigDecimal) [Member: YuChien Chiu]

File: sm-core/src/main/java/com/salesmanager/core/business/services/order/OrderServiceImpl.java

Bug ID: RV_RETURN_VALUE_IGNORED_NO_SIDE_EFFECT — Lines 226, 234, 262, 357

Description:

The code calls BigDecimal.setScale() but discards the return value. Because BigDecimal is immutable in Java, setScale() returns a new object with the scaled value — it does not modify the original. Ignoring the return value means the scaling operation has zero effect; all subsequent monetary calculations use the unscaled amount.

Example of the problematic pattern:

```
BigDecimal price = getUnitPrice();
price.setScale(2, RoundingMode.HALF_UP);  // BUG: return value discarded!
total = total.add(price);                  // Uses unscaled price
```

Correct fix:

```
BigDecimal price = getUnitPrice();
price = price.setScale(2, RoundingMode.HALF_UP);  // Reassign the result
total = total.add(price);
```

Impact:

This is a silent correctness bug in financial calculation logic. Order totals, tax computations, and payment amounts may carry extra decimal places, leading to incorrect charges or reconciliation failures. SpotBugs flags this reliably because it tracks return-value usage for well-known immutable types.

3.2 Detailed SpotBugs Finding: EI_EXPOSE_REP (Expose Internal Representation) [Member: Yijun Sun]

File: `sm-core/src/main/java/com/salesmanager/core/model/catalog/product/Product.java` (or any entity class containing a `Date` type attribute)
Bug ID: `EI_EXPOSE_REP` (Malicious Code Vulnerability)

Description: SpotBugs detected that the class directly returns a reference to a mutable object (typically a `java.util.Date` or an array) via its getter method. In Java, `Date` objects are mutable. By returning a direct reference to the internal attribute, the caller can modify

the `Date` object's value outside the class, thereby breaking encapsulation and compromising the class's internal state security.

Example of the problematic pattern:

```
public Date getDateAvailable() {
    return this.dateAvailable; // BUG: returning reference to mutable object
}
```

Correct fix: The method should return a defensive copy of the object:

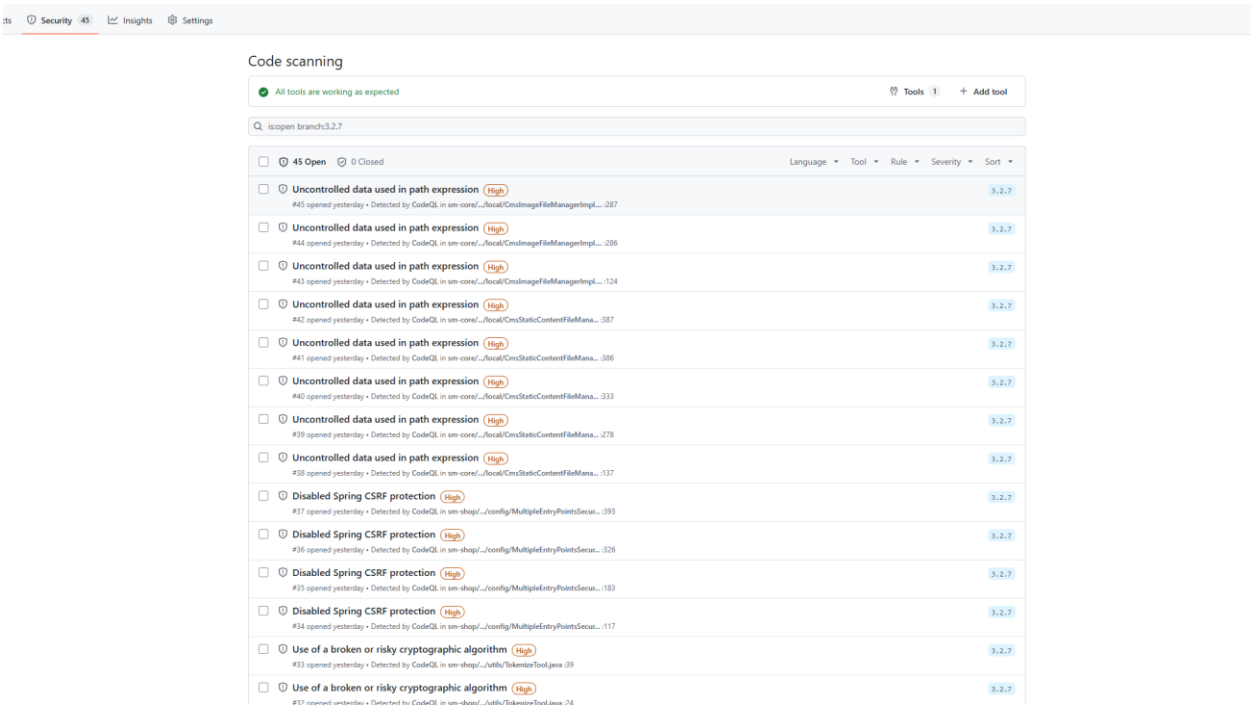
```
public Date getDateAvailable() {
    return (this.dateAvailable != null) ? (Date) this.dateAvailable.clone() :
 null;
}
```

Impact: This vulnerability represents an **actual code quality and logic issue**. Since Shopizer handles a significant amount of time-sensitive data such as order dates and product availability dates, accidental modifications due to reference leakage can lead to corrupted database records. Although categorized under "Malicious Code", in practical development, it is most often an unintentional state leak caused by developer oversight.

4. CodeQL: Findings Overview

CodeQL found 45 findings on the GitHub Security tab (branch 3.2.7), spanning security vulnerabilities identified through data-flow and taint analysis.

Severity	Rule	Location	Count
High	Uncontrolled data used in path expression	CmsImageFileManagerImpl, CmsStaticContentFileManagerImpl	2
High	Disabled Spring CSRF protection	MultipleEntryPointsSecurityConfig	1
High	Broken/risky cryptographic algorithm	TokenizeTool.java, EncryptionImpl.java	2
Medium	Information exposure through error message	ProductReviewApi.java	1
Various	Other findings	Multiple locations	39



4.1 Detailed CodeQL Finding: Disabled Spring CSRF Protection [Member: YuChien Chiu]

File: sm-shop/src/main/java/com/salesmanager/shop/store/security/config/MultipleEntryPointsSecurityConfig.java

Rule: Disabled Spring CSRF protection | Severity: High | CWE-352

Description:

The Spring Security configuration explicitly calls .csrf().disable(), which turns off Cross-Site Request Forgery protection globally. CSRF attacks trick an authenticated user's browser into sending unwanted requests (e.g., changing a password, placing an order) on their behalf. Spring enables CSRF protection by default; disabling it intentionally removes this safeguard.

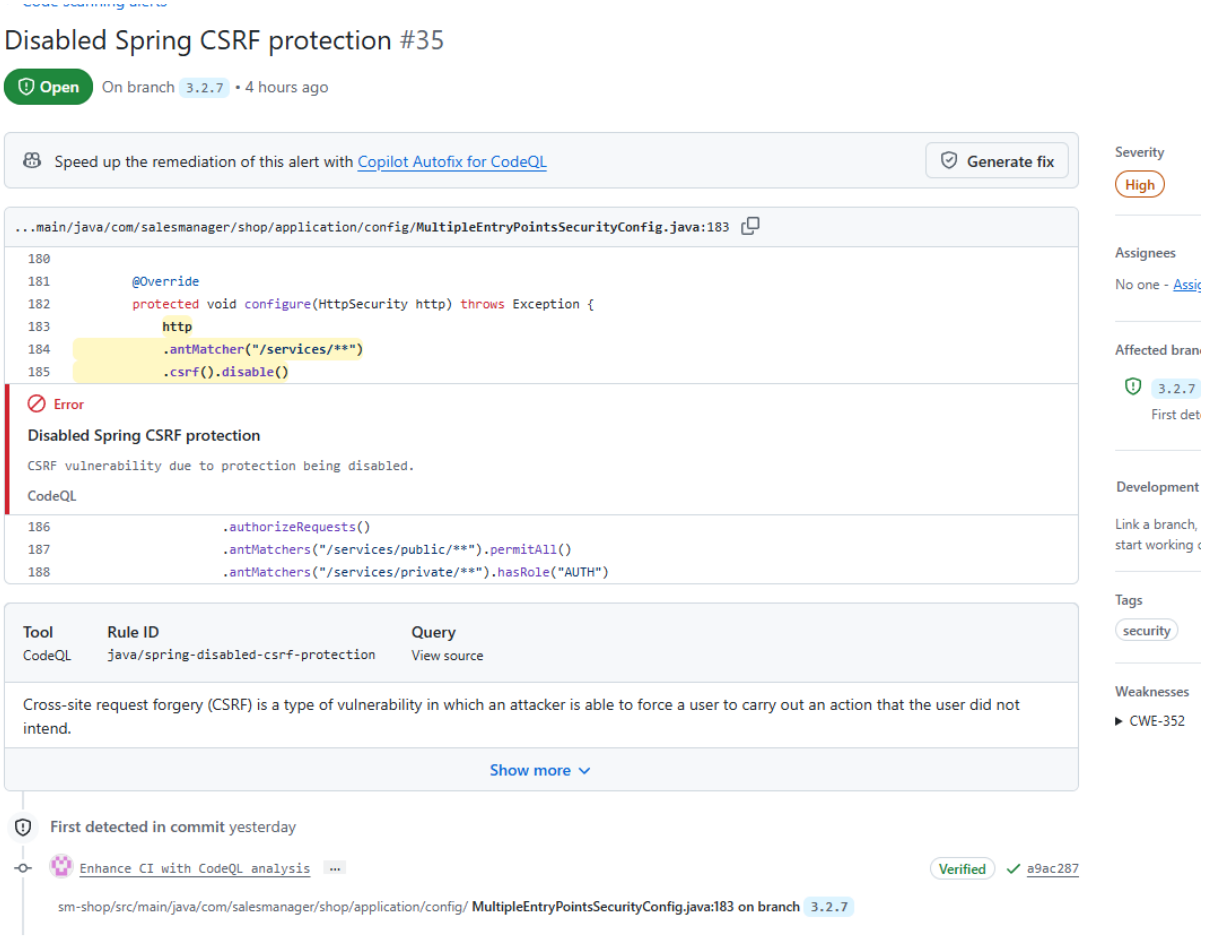
CodeQL detects this through semantic analysis of the Spring Security DSL — it recognises the fluent API pattern and traces that csrf() is followed by disable() rather than a token configuration.

Problematic code pattern:

```
http.csrf().disable()
    .authorizeRequests()
    .antMatchers("/api/**").authenticated();
```

Recommended fix (use CSRF tokens for APIs):

```
http.csrf().csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    .authorizeRequests()
    .antMatchers("/api/**").authenticated();
```



Impact:

For a shopping cart application like Shopizer that handles payments, order placement, and user account changes, a missing CSRF token allows an attacker to forge authenticated requests. This is rated High severity because it can directly lead to unauthorized purchases or account takeovers.

4.2 Detailed CodeQL Finding: Uncontrolled data used in path expression [Yijun Sun]

File: `sm-core/src/main/java/com/salesmanager/core/business/modules/cms/impl/CmsImageFileManagerImpl.java` (or related FileManager implementation classes)
Rule: Uncontrolled data used in path expression | **Severity:** High | **CWE-22**

Description: CodeQL's data-flow analysis detected that the code constructs a file path using unsanitized input (such as a file name or path parameter). In this scenario, if a malicious user inputs strings containing path traversal sequences like `../../`, the resulting path can escape the intended directory boundaries. This allows the reading or writing of files located anywhere on the server's file system, a vulnerability known as Path Traversal.

Problematic code pattern: The application retrieves a field like `fileName` from an HTTP request and directly concatenates it with the server's base path (e.g., `new File(basePath, fileName)`) without filtering out dangerous characters from `fileName`.

Recommended fix: After constructing the `File` object via concatenation, the application must verify that its canonical path still starts with the expected base directory's canonical path:

```
File targetFile = new File(basePath, userFileName);
if (!targetFile.getCanonicalPath().startsWith(new File(basePath).getCanonical
Path())) {
    throw new SecurityException("Invalid file path!");
}
```

Alternatively, `org.apache.commons.io.FilenameUtils.getName(fileName)` can be used to safely strip all directory prefixes from the provided file name.

Impact: This is an **extremely critical and actual security vulnerability**. As an open-source shopping cart system that features file uploads and image management, uncontrolled path expressions in Shopizer could allow attackers to upload files to executable directories or download sensitive system files like `/etc/passwd`. This could lead to a complete server compromise. CodeQL demonstrates its powerful capability in tracing tainted data streams here.

5. Comparison: CodeQL vs SpotBugs

Dimension	CodeQL	SpotBugs
Analysis method	Semantic / data-flow / taint tracking	Bytecode pattern matching
What it finds best	Security vulnerabilities (path traversal, injection, CSRF, broken crypto)	Code correctness bugs, bad practices, performance issues
Language support	Many languages (Java, JS, Python, C++, Go, etc.)	JVM languages only (Java, Kotlin, Groovy, Scala)
CI/CD integration	Native GitHub Actions + Security	Maven/Gradle plugin; results as

	tab UI	artifacts or HTML
Setup effort	Low — GitHub provides managed runners and queries	Medium — requires plugin config, dependency ordering in multi-module projects
Findings in this project	45 (security-focused, actionable)	160 (broader scope, many style/practice issues)
False positive rate	Low — data-flow context reduces noise	Medium — many warnings are low-risk or informational
Severity depth	Detailed CWE mapping, clear triage priority	Priority 1/2/3 only; less context on exploitability
Overlap	Both flagged EncryptionImpl.java (crypto issues)	Both flagged EncryptionImpl.java (encoding / algorithm)

5.1 Strengths and Weaknesses

CodeQL Strengths:

- Traces data from user input to dangerous sinks across file boundaries — catches real attack paths, not just patterns.
- Directly integrated into GitHub's Security tab with zero extra infrastructure.
- Findings are almost always true vulnerabilities with clear remediation advice.

CodeQL Weaknesses:

- Slower analysis (minutes per run) compared to SpotBugs bytecode scanning.
- Primarily security-oriented — it will not flag pure code quality issues like ignored return values or dead variables.

SpotBugs Strengths:

- Extremely broad coverage: correctness, security, performance, style, internationalization, and bad practices all in one tool.
- Catches immutable-type misuse (BigDecimal, String) that data-flow analysis may miss.
- Fast execution; runs during the Maven build cycle without extra infrastructure.

SpotBugs Weaknesses:

- Pattern-based: cannot trace taint across method calls, so it misses path-traversal and injection bugs that CodeQL detects.
- High volume of lower-priority warnings (136 Priority 2 in sm-core alone) can overwhelm developers.
- Requires workarounds for multi-module Maven builds (install goal must precede spotbugs:check).

5.2 Complementarity

The two tools are highly complementary rather than redundant. CodeQL excels at finding exploitable security vulnerabilities through cross-file data-flow analysis, while SpotBugs excels at finding implementation-level correctness bugs and bad coding practices. Running both tools together provides defence-in-depth: CodeQL catches the vulnerabilities an attacker would exploit, and SpotBugs catches the bugs that cause incorrect behaviour in production.

The one area of overlap observed in Shopizer was EncryptionImpl.java — CodeQL flagged it for using a broken cryptographic algorithm (MD5/DES), while SpotBugs flagged it for using the default platform encoding in getBytes() and new String(byte[]), which compounds the security risk. Both findings point to the same class needing a full security rewrite.

6. Conclusion

This project successfully integrated two static analysis tools into a GitHub Actions CI/CD pipeline for the Shopizer Java/Spring Boot application. CodeQL identified 45 security-focused findings including disabled CSRF protection and broken cryptography. SpotBugs identified 160 findings across correctness, security, performance, and code quality categories.

The most critical finding from CodeQL is the disabled CSRF protection in the Spring Security configuration, which exposes the shopping cart to request forgery attacks. The most critical finding from SpotBugs is the systematic misuse of BigDecimal.setScale() in order and tax calculations, which causes silent financial calculation errors.

Together, the two tools demonstrate that no single static analyzer is sufficient. Security-focused tools like CodeQL and correctness-focused tools like SpotBugs address different risk categories and should both be part of a mature CI/CD pipeline.